Algorithms

Week 11 – Graph Algorithms I

Korea University Sejong Campus, Dept. of Computer Science & Software

Part 1. Graph Basics and Representations

Learning Objectives

- Understand graph representations: adjacency matrix, adjacency list
- Deeply understand BFS and DFS traversal principles
- Learn the meaning of spanning trees and two MST algorithms
- Understand topological sorting and its applications to DAGs
- Analyze the running time of each algorithm

Textbook: CLRS 3rd Edition, Chapters 22-23

What is a Graph?

> A graph models relationships between objects using **vertices** and **edges**.

- Graph **G = (V, E)**
  - **V**: set of vertices (nodes)
  - **E**: set of edges (connections)
- Two vertices connected by an edge are called **adjacent**

```
1 --- 2
|   / |
|  /  |
| /   |
3 --- 4
```

Applications: social networks, road maps, circuit design, web page links, task scheduling, …

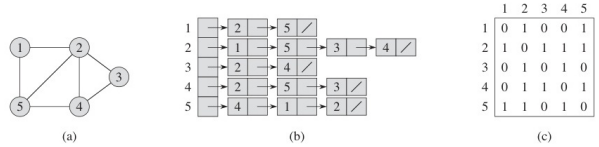
Types of Graphs

Type	Description	Example
Undirected	Edges have no direction	Friendship network
Directed (Digraph)	Edges have direction (arrows)	Web page links
Weighted	Edges carry numerical weights	Road distances
Unweighted	All edges are equal	Simple connectivity

```
Undirected:          Directed:          Weighted:
  1 --- 2            1 ⟶ 2              1 --5-- 2
  |     |            |     ^              |       |
  3 --- 4            v     |              3 --2-- 4
                     3 ⟶ 4                  3
```

Graph Representation 1: Adjacency Matrix

- Use an **N x N** matrix (N = number of vertices)
  - Entry (i, j) = **1** if edge exists; **0** if not
- **Directed graph**: entry (i, j) means edge from i to j
- **Weighted graph**: entry (i, j) stores the **weight** instead of 1




```
Graph:          Adjacency Matrix:
  1 --- 2          1  2  3  4
  |   / |      1 [ 0  1  1  0 ]
  |  /  |      2 [ 1  0  1  1 ]
  | /   |      3 [ 1  1  0  1 ]
  3 --- 4      4 [ 0  1  1  0 ]
```

Adjacency Matrix – Directed Example

```
Directed Graph:        Adjacency Matrix:
  1 ⟶ 2                    1  2  3  4
  |     ^              1 [ 0  1  1  0 ]
  v     |              2 [ 0  0  0  0 ]
  3 ⟶ 4                3 [ 0  0  0  1 ]
                       4 [ 0  1  0  0 ]
```

Note: The matrix is **not symmetric** for directed graphs.

Adjacency Matrix – Weighted Example

```
Weighted Graph:        Adjacency Matrix:
  1 --5-- 2                1   2   3   4
  |       |            1 [ 0   5   3   0 ]
  3       7            2 [ 5   0   0   7 ]
  |       |            3 [ 3   0   0   2 ]
  3 --2-- 4            4 [ 0   7   2   0 ]
```

- Use **0** or **infinity** to indicate no edge (convention varies)

Adjacency Matrix – Analysis

Advantages:

- Easy to understand and implement
- **O(1)** time to check if edge (i, j) exists

Disadvantages:

- Requires **O(N^2)** space regardless of edge count
- Matrix initialization takes **O(N^2)** time
- Inefficient for **sparse** graphs

Example: 1,000,000 vertices with 2,000,000 edges

- Matrix has **10^12** entries, but only 2,000,000 are non-zero
- Massive waste of space!

> **Best for:** Dense graphs where most vertex pairs have edges

Graph Representation 2: Adjacency List

- Use **N linked lists** (one per vertex)
- The i-th list contains all vertices adjacent to vertex i
- For **weighted** graphs, each list node also stores the edge weight

```
Graph:              Adjacency List:
  1 --- 2           1: [2] → [3]
  |   / |           2: [1] → [3] → [4]
  |  /  |           3: [1] → [2] → [4]
  | /   |           4: [2] → [3]
  3 --- 4
```

Adjacency List – Weighted Example

```
Weighted Graph:     Adjacency List (vertex, weight):
  1 --5-- 2         1: [(2,5)] → [(3,3)]
  |       |         2: [(1,5)] → [(4,7)]
  3       7         3: [(1,3)] → [(4,2)]
  |       |         4: [(2,7)] → [(3,2)]
  3 --2-- 4
```

Adjacency List – Analysis

Advantages:

- Space proportional to edges: **O(V + E)**
- No wasted space – especially useful when edges are few
- Efficient for iterating over neighbors of a vertex

Disadvantages:

- Checking if edge (i, j) exists takes **O(degree(i))** time – not O(1)
- When the graph is very dense, the linked list overhead can be a problem

Best for: Sparse graphs where E is much smaller than V^2

Graph Representation 3: Adjacency Array

- Replace linked lists with **arrays** for each vertex
- Store a count of adjacent vertices and an array of neighbors
- Can also be packed into a **single contiguous array** with an index table

```
Adjacency Array:
Vertex:  1    2    3    4
Count:  [2]  [2]  [3]  [2]

1: [2, 3]
2: [1, 3, 4]         (packed into one array)
3: [1, 2, 4]         A = [2,3 | 1,3,4 | 1,2,4 | 2,3]
4: [2, 3]            Index: [0, 2, 5, 8, 10]
```

Advantages over adjacency list:

- Less pointer overhead, better cache performance
- Can use **binary search** on sorted neighbor arrays: O(log(degree))

Representation Comparison

	Adjacency Matrix	Adjacency List	Adjacency Array
Space	O(V^2)	O(V + E)	O(V + E)
Edge lookup	O(1)	O(degree)	O(log degree)
Iterate neighbors	O(V)	O(degree)	O(degree)
Best for	Dense graphs	Sparse graphs	Sparse, static graphs

In practice, **adjacency lists** (or arrays) are used most often because real-world graphs tend to be sparse.

Part 2. Graph Traversal: BFS and DFS

Visiting All Vertices in a Graph

Two fundamental approaches:

	BFS (Breadth-First Search)	**DFS** (Depth-First Search)
Strategy	Visit level by level	Go as deep as possible first
Data structure	Queue	Stack (or recursion)
Time complexity	O(V + E)	O(V + E)

```
        1
       /|\
      2  3  4
     /|     |
    5  6    7

BFS from 1: 1 → 2, 3, 4 → 5, 6, 7
DFS from 1: 1 → 2 → 5 → 6 → 3 → 4 → 7
```

These are the **foundation** of all graph algorithms. Understanding BFS and DFS deeply is essential for mastering graph algorithms.

DFS – Algorithm (Pseudocode)

```
DFS(G)
    for each v in V
        visited[v] ← NO
    for each v in V
        if visited[v] = NO then aDFS(v)

aDFS(v)
    visited[v] ← YES                    ... (A)
    for each x in L(v)                  ... (B)
        if visited[x] = NO then aDFS(x)  ... (C)
```

- **(A)** Mark current vertex as visited: total **O(V)** across all calls
- **(B)** Scan adjacency list of v
- **©** Recursive call only if not visited: each edge examined at most twice (once from each endpoint in undirected) – total **O(E)**

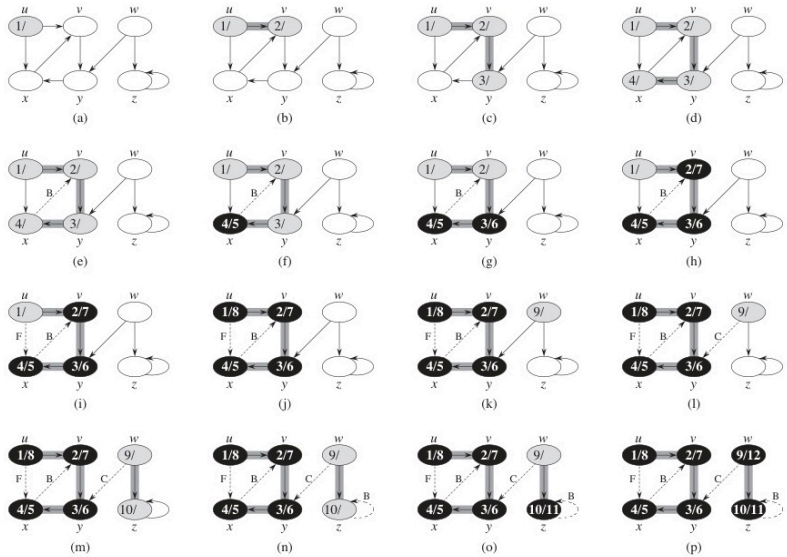
Time complexity: O(V + E)

DFS – Step-by-Step Example

```
Graph:                          Adjacency Lists:
  1 --- 2                       1: [2, 3, 4, 6]
  |   / | \                     2: [1, 3]
  |  /  |  6                    3: [1, 2, 5]
  | /   |                       4: [1, 6]
  3     5                       5: [3, 6]
  |   /                         6: [1, 4, 5]
  4--6
```

```
DFS starting from vertex 1:
Step 1: Visit 1, go to neighbor 2
Step 2: Visit 2, go to neighbor 3
Step 3: Visit 3, go to neighbor 5
Step 4: Visit 5, go to neighbor 6
Step 5: Visit 6, go to neighbor 4
Step 6: Visit 4, backtrack

Traversal: 1→2→3→5→6→4
```



DFS – Detailed Trace

```
Call stack trace:

aDFS(1)                     visited: {1}
  aDFS(2)                   visited: {1,2}
    aDFS(3)                 visited: {1,2,3}
      aDFS(5)               visited: {1,2,3,5}
        aDFS(6)             visited: {1,2,3,5,6}
          aDFS(4)           visited: {1,2,3,5,6,4}
            neighbors: 1(visited), 6(visited)
            → return
          → return
        → return
      → return
    → return
  → return
```

Each vertex is visited **exactly once**. Each edge is checked **at most twice**.

BFS – Algorithm (Pseudocode)

```
BFS(G, s)
    for each v in V - {s}
        visited[v] ← NO
    visited[s] ← YES
    enqueue(Q, s)                        ... (A)
    while Q is not empty
        u ← dequeue(Q)                   ... (A)
        for each v in L(u)              ... (B)
            if visited[v] = NO then
                visited[v] ← YES
                enqueue(Q, v)
```

- **(A)** Each vertex enters and leaves the queue exactly once: total **O(V)**
- **(B)** Each adjacency list is scanned once: total **O(E)**

Time complexity: O(V + E)

BFS – Step-by-Step Example

```
Graph:                      Adjacency Lists:
  1 --- 2                   1: [2, 3, 4, 6]
  |   / | \                 2: [1, 3]
  |  /  |  6                3: [1, 2, 5]
  | /   |                   4: [1, 6]
  3     5                   5: [3, 6]
  |   /                     6: [1, 4, 5]
  4--6
```

```
BFS starting from vertex 1:

Step 1: Dequeue 1.  Visit neighbors: 2, 3, 4, 6
        Queue: [2, 3, 4, 6]       Level 0: {1}

Step 2: Dequeue 2.  Neighbor 1(visited), 3(visited) → skip
        Queue: [3, 4, 6]          Level 1: {2, 3, 4, 6}

Step 3: Dequeue 3.  Neighbor 5 not visited → enqueue
        Queue: [4, 6, 5]

Step 4: Dequeue 4.  All neighbors visited → skip
        Queue: [6, 5]

Step 5: Dequeue 6.  All neighbors visited → skip
```

BFS Gives Shortest Hop-Count Paths

Starting from vertex 1, BFS naturally discovers vertices **level by level**:

```
Level 0: {1}           ← 0 hops from source
Level 1: {2, 3, 4, 6}  ← 1 hop from source
Level 2: {5}           ← 2 hops from source
```

> In an **unweighted** graph, BFS finds the **shortest path** (minimum number of edges) from the source to every reachable vertex.

This is one of the most important properties of BFS.

BFS vs DFS – Comparison

Property	BFS	DFS
Data structure	Queue (FIFO)	Stack / Recursion (LIFO)
Order	Level by level	As deep as possible
Shortest path	Yes (unweighted)	No
Memory	O(V) for the queue	O(V) for the recursion stack
Time	O(V + E)	O(V + E)
Common uses	Shortest path, level-order	Topological sort, cycle detection, SCC

Both have the same time complexity, but they explore the graph in fundamentally different orders.

Part 3. Topological Sorting

Topological Sorting

Precondition: Directed Acyclic Graph (DAG) – no cycles

Definition:

- Arrange all vertices in a line such that for every directed edge (x, y), vertex x appears **before** vertex y
- Multiple valid orderings may exist for a given DAG

```
DAG:
  A ⟶ B ⟶ D
  |         ^
  v        /
  C ———————

Topological orders:
  A, B, C, D    (valid)
  A, C, B, D    (valid)
```

Use case: Task scheduling where some tasks must precede others (e.g., course prerequisites, build systems, recipe steps)

Topological Sort – Algorithm 1 (In-Degree)

```
topologicalSort1(G)
    for i ← 1 to n
        Select a vertex u with no incoming edges (in-degree = 0)
        A[i] ← u
        Remove vertex u and all its outgoing edges from the graph
    // A[1..n] now contains the topological order
```

Time complexity: O(V + E)

```
Example (ramen recipe):

Step (a): "Open packet" has in-degree 0 → select it, remove edges
Step (b): "Boil water" has in-degree 0 → select it, remove edges
Step (c): "Add noodles" has in-degree 0 → select it, remove edges
  ... continue until all vertices are placed
```

Topological Sort – Algorithm 2 (DFS-Based)

```
topologicalSort2(G)
    for each v in V
        visited[v] ← NO
    for each v in V
        if visited[v] = NO then DFS-TS(v)

DFS-TS(v)
    visited[v] ← YES
    for each x in L(v)
        if visited[x] = NO then DFS-TS(x)
    Insert v at the FRONT of linked list R    // after visiting all children
```

- When DFS finishes processing a vertex (all descendants done), insert it at the **front** of the result list
- After the algorithm completes, list R contains the topological order

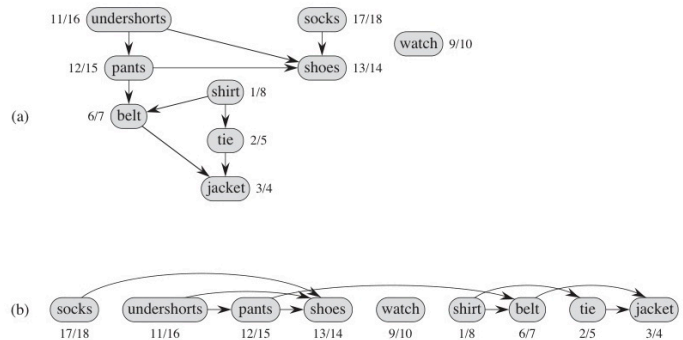
Time complexity: O(V + E) (same as DFS)

Topological Sort – DFS Example

```
DAG:
  A ⟶ B ⟶ D
  |         ^
  v        /
  C ──────

DFS-TS from A:
  Visit A → Visit B → Visit D
    D done → R = [D]
    B done → R = [B, D]
  Visit C
    D already visited → R = [C, B, D]
  A done → R = [A, C, B, D]

Result: A → C → B → D  ✓
```



Key insight: A vertex is added to the front **only after** all reachable vertices have been processed.

Part 4. Minimum Spanning Trees

Minimum Spanning Tree (MST) – Concept

Prerequisites:

- **Connected graph**: a path exists between every pair of vertices
- **Undirected** and **weighted** edges

Tree:

- A connected graph with **no cycles**
- A tree with n vertices always has exactly **n - 1** edges

Spanning tree of G:

- A tree that includes **all vertices** of G, using only edges from G

Minimum Spanning Tree:

- The spanning tree with the **minimum total edge weight**

MST – Motivation

```
Problem: Connect all cities with minimum road construction cost.

Cities (vertices):       Possible road costs (edges):
  A --5-- B              Spending on ALL roads: very expensive
  |\ /|                  Goal: connect all cities using
  | X  |                      the FEWEST, CHEAPEST roads
  |/ \|
  C --3-- D

MST solution: select n-1 edges with minimum total weight
              such that all vertices remain connected
```

MST algorithms are **greedy** algorithms that guarantee an **optimal** solution – a rare and valuable property of greedy approaches.

Prim's Algorithm – Idea

Strategy: Grow the MST one vertex at a time from a starting vertex.

```
Prim(G, r)
    S ← empty set
    Mark vertex r as visited; add r to S
    while S ≠ V
        Find the minimum-weight edge (x, y) connecting
            S to V-S  (x in S, y in V-S)
        Mark y as visited; add y to S
```

- At each step, pick the **cheapest edge** crossing the cut (S, V-S)
- This is a **greedy** choice – and it provably yields the optimal MST

Time complexity: O(E log V) using a binary heap

Prim's Algorithm – Detailed Pseudocode

```
Prim(G, r)                          // G=(V,E), r=start vertex
    S ← empty
    for each u in V
        d[u] ← infinity             // (1) Initialize distances
    d[r] ← 0                        // (2) Start vertex distance = 0
    while S ≠ V                     // (3) Repeat |V| times
        u ← extractMin(V-S, d)      // (4) Get closest vertex
        S ← S + {u}                 // (5) Add to MST set
        for each v in L(u)          // (6) Check neighbors
            if v in V-S and w(u,v) < d[v] then
                d[v] ← w(u,v)       // (7) Relaxation
                tree[v] ← u         // Remember parent
```

- **extractMin** uses a **heap** -> O(log V) per extraction
- **Relaxation** (line 7): update d[v] if a cheaper connection to S is found
- Total: **O(E log V)**

Prim's Algorithm – Step-by-Step Example

```
Weighted Graph:
        8
    A ———————— B
    |\        /|
    | \5   9/  |
   3|  \  /   7|
    |   \/     |
    |   /\     |
    |  /  \    |
    | /  2 \   |
    C————————D
       |   /
      4|  /6
       | /
       E

Edges: (A,B,8) (A,C,3) (A,D,5) (B,D,9) (B,E,7) (C,D,2) (C,E,4) (D,E,6)
```

Prim's – Execution Trace (start at A)

```
Step 0: S={A}     d: A=0, B=inf, C=inf, D=inf, E=inf
        Edges from A: (A,B,8), (A,C,3), (A,D,5)
        Update: d[B]=8, d[C]=3, d[D]=5

Step 1: extractMin → C (d=3).  S={A,C}
        Edges from C: (C,D,2), (C,E,4)
        Update: d[D]=min(5,2)=2, d[E]=4
        MST edge: A--C (weight 3)

Step 2: extractMin → D (d=2).  S={A,C,D}
        Edges from D: (D,B,9), (D,E,6)
        Update: d[B]=min(8,9)=8 (no change), d[E]=min(4,6)=4 (no change)
        MST edge: C--D (weight 2)

Step 3: extractMin → E (d=4).  S={A,C,D,E}
        Edges from E: (E,B,7)
        Update: d[B]=min(8,7)=7
        MST edge: C--E (weight 4)

Step 4: extractMin → B (d=7).  S={A,C,D,E,B}
        MST edge: E--B (weight 7)

MST total weight: 3 + 2 + 4 + 7 = 16
```

Prim's – MST Result

```
Original Graph:                MST (weight = 16):
        8
    A ———————— B                   A         B
    |\        /|                   |         |
    | \5   9/  |                   |3        |7
    |  \  /   7|                   |         |
    |   \/     |                   C         |
    |   /\     |                   |\        |
    |  /  \    |                   | \2      |
    | /  2 \   |                   |  D      |
    C————————D                     |
       |   /                      4|
      4|  /6                       |
       | /                         E
       E

MST edges: A-C(3), C-D(2), C-E(4), E-B(7)
Total: 16
```

Kruskal's Algorithm – Idea

Strategy: Sort all edges by weight. Add edges one by one, skipping any edge that would create a cycle.

```
Kruskal(G)
    T ← empty set                      // MST edges
    Create n singleton sets (one per vertex)
    Sort all edges Q by weight (ascending)
    while |T| < n-1
        Remove minimum-weight edge (u, v) from Q
        if u and v are in DIFFERENT sets then
            Union the two sets
            T ← T + {(u, v)}
```

- **Cycle detection** uses **Union-Find** (disjoint set) data structure
  - Find-Set: O(log V) – check which set a vertex belongs to
  - Union: O(log V) – merge two sets
- Edge sorting: O(E log E) = O(E log V)

Time complexity: O(E log V)

Kruskal's – Step-by-Step Example

```
Edges sorted by weight:
  (C,D,2), (A,C,3), (C,E,4), (A,D,5), (D,E,6), (B,E,7), (A,B,8), (B,D,9)

Initial sets: {A}, {B}, {C}, {D}, {E}
```

```
Step 1: Edge (C,D,2) -- C and D in different sets → ADD
        Sets: {A}, {B}, {C,D}, {E}        T = {(C,D)}

Step 2: Edge (A,C,3) -- A and C in different sets → ADD
        Sets: {A,C,D}, {B}, {E}           T = {(C,D),(A,C)}

Step 3: Edge (C,E,4) -- C and E in different sets → ADD
        Sets: {A,C,D,E}, {B}              T = {(C,D),(A,C),(C,E)}

Step 4: Edge (A,D,5) -- A and D in SAME set → SKIP (would create cycle)

Step 5: Edge (D,E,6) -- D and E in SAME set → SKIP

Step 6: Edge (B,E,7) -- B and E in different sets → ADD
        Sets: {A,B,C,D,E}                 T = {(C,D),(A,C),(C,E),(B,E)}
        |T| = 4 = n-1 → DONE!

MST total weight: 2 + 3 + 4 + 7 = 16
```

Kruskal's – MST Result

```
Edges considered:                MST (weight = 16):
  (C,D,2)  → ADDED                   A         B
  (A,C,3)  → ADDED                   |         |
  (C,E,4)  → ADDED                 3|        7|
  (A,D,5)  → SKIPPED (cycle)         |         |
  (D,E,6)  → SKIPPED (cycle)         C─────────E
  (B,E,7)  → ADDED                   |       4
                                   2|
                                     |
                                     D

Same MST as Prim's: A-C(3), C-D(2), C-E(4), E-B(7)
Total: 16
```

Both Prim's and Kruskal's produce the same MST (when the MST is unique).

Prim's vs Kruskal's

	Prim's	Kruskal's
Strategy	Grow MST from one vertex	Pick cheapest edge globally
Data structure	Priority queue (heap)	Union-Find + sorted edges
Time	O(E log V)	O(E log V)
Better when	Dense graphs (E close to V^2)	Sparse graphs (E close to V)
Approach	Vertex-centric	Edge-centric

Both are **greedy** algorithms that guarantee the **optimal** MST.

Summary

Topic	Key Points
Graph	G = (V, E); directed/undirected; weighted/unweighted
Adjacency Matrix	O(V^2) space, O(1) edge lookup; best for dense graphs
Adjacency List	O(V+E) space, O(degree) lookup; best for sparse graphs
DFS	Recursive / stack-based; O(V+E); goes deep first
BFS	Queue-based; O(V+E); level by level; shortest hop-count paths
Topological Sort	DAG only; O(V+E); in-degree method or DFS-based
Prim's MST	Grow from a vertex; extractMin with heap; O(E log V)

Q & A

codingchild@korea.ac.kr